Данила Аринов

Junior-разработчик — очереди и фоновые задачи

+7 901 431-82-98 · teivrim@gmail.com · Telegram @Smishnyavko · github.com/TeivrimOriginal · Портфолио · LinkedIn

О СЕБЕ

Фоновые задачи на Celery с Redis в качестве брокера. Отдельно сделал так, чтобы тесты не зависели от живого воркера — задачи выполняются синхронно, и проверять их поведение можно обычным тестом. **288 тестов TaskHub проходят против живой БД.**

ОПЫТ РАБОТЫ

Фриланс — заказы на Kwork

`Python · по заказам на Kwork`

- Telegram-боты, парсеры сайтов, REST API, интеграции внешних API, автоматизация по ТЗ заказчиков; kwork.ru/user/teivrim.

ПРОЕКТЫ

TaskHub API — сервис управления задачами

`FastAPI · PostgreSQL 17.4 · Celery · Redis`

- Асинхронный сервис на FastAPI и SQLAlchemy 2.0: `asyncpg`, Pydantic v2, разделение схем запросов и ответов, OpenAPI 3.1 из коробки.
- Аутентификация JWT с ротацией refresh-токенов и RBAC: роли проверяются на уровне зависимости, доступ к эндпоинту нельзя получить без проверки прав.
- Фоновые задачи на Celery, Redis как брокер и кэш; задачи выполняются синхронно при `CELERY_TASK_ALWAYS_EAGER`, поэтому тесты не зависят от воркера.
- 8 таблиц по схеме Alembic, `citext` и `pg_trgm` через расширения PostgreSQL.
- **288 тестов проходят против живой PostgreSQL 17.4**, 6 пропущено — только Redis-специфичные. Покрытие 90,24% с учётом ветвлений при гейте 85%.

github.com/TeivrimOriginal/taskhub-api — репозиторий открыт, тесты можно гонять

anime-sync-api — REST API каталога аниме

`FastAPI · PostgreSQL 16 · Alembic · GIN`

- Асинхронный стек: FastAPI, SQLAlchemy 2.0 (`asyncpg`), Pydantic v2, httpx. Схемы запросов и ответов разделены.
- Три таблицы с внешними ключами и CHECK-ограничениями; `UNIQUE (source, external_id)` — внешний id уникален в пределах источника, потому что у AniList и Jikan нумерация разная.

github.com/TeivrimOriginal/anime-sync-api

НАВЫКИ

Очереди и кэш: Celery, Redis как брокер и кэш, TTL, eager-режим для тестов

Асинхронность: asyncio, httpx, asyncpg, конкурентность против параллельности, пул соединений

SQL и PostgreSQL: PostgreSQL 16/17 (tsvector, GIN, pg_trgm, CHECK, FK CASCADE, EXPLAIN, индексы), Alembic-миграции, SQLite, JOIN/GROUP BY, запросы к регистрам

Тесты: pytest, pytest-asyncio, httpx ASGITransport, фикстуры, моки внешних API, Allure, Selenium 4, покрытие с ветвлениями

ДОПОЛНИТЕЛЬНО

Понимаю, что Celery — это про очереди и повторяемость, а не про «просто поставить задачу в фон». Поэтому отказ пишется в базу, а не теряется. Английский — чтение технической документации. Открытый код: 12 активных публичных репозиториев, у каждого проекта тесты и CI.